Task 1 Report: Lookup and Insertion Performance of BTree, DynamicPGM, and LIPP

Prepared from committed benchmark artifacts in the repository

March 25, 2026

Abstract

This report documents Task 1 of the COS568 Learned Index project using the benchmark artifacts committed in this repository. The task asks us to compare the lookup and insertion behavior of three indexes—BTree, DynamicPGM, and LIPP—on the Facebook, Books, and OSMC 100M uint64 datasets under four prescribed workloads. We summarize what the assignment required, what was executed for this milestone, how configurations were selected from the raw CSV outputs, and what the recorded results show about throughput and index size. Across the three datasets, LIPP is overwhelmingly best for read-heavy workloads, DynamicPGM is best for insertion-heavy workloads, and BTree is the most stable but never the top performer on either extreme. That performance split is paired with a large size tradeoff: DynamicPGM is the most compact, BTree is modestly larger, and LIPP uses much more memory in every dataset and workload.

1 Assignment context and Task 1 objective

The project studies *learned indexes*: data structures that model the mapping from keys to positions in sorted data rather than relying only on classical tree navigation. The repository README frames two milestones. Task 1 is the measurement and comparison baseline; Task 2 then builds a hybrid design on top of the insights from Task 1.

For Task 1, the assignment requires the following:

- understand the codebase and the benchmark pipeline in `scripts/`,
- evaluate lookup and insertion performance of BTree, DynamicPGM, and LIPP,
- use the Facebook, Books, and OSMC 100M uint64 datasets,
- run the provided workloads with three repetitions each,
- compare average throughput across methods and explain the observed differences,
- choose the best hyperparameter setting for BTree and DynamicPGM for each dataset/workload pair.

The four workloads described in the assignment are:

1. **Lookup-only**: 2M lookups with a 50% negative lookup ratio.

2. **Insert-Lookup (50/50)**: 1M inserts followed by 1M lookups, with 50% negative lookups in the lookup phase. The benchmark reports both insert throughput and post-insert lookup throughput.
3. **Mixed workload with 10% inserts**: insertions and lookups are mixed, and the benchmark reports a single mixed throughput value.
4. **Mixed workload with 90% inserts**: again mixed, but insertion-heavy.

The assignment also notes that BTree and DynamicPGM are run with multiple hyperparameter settings. For each dataset/workload pair, the best-performing row is the one with the highest average throughput across the three repeats.

2 What we did for Task 1

The committed summary file (`TASK1_SUMMARY.txt`) records the Task 1 workflow that was carried out:

- downloaded the Facebook, Books, and OSMC 100M uint64 datasets,
- generated the four Task 1 workloads for each dataset,
- built the benchmark binary,
- ran the full sweep of 3 datasets $\times$ 3 indexes $\times$ 4 workloads with 3 repeats each,
- selected, for each index and workload, the row with the highest average throughput across those three runs.

The benchmark outputs are stored in `results/*.csv`, and previous aggregated throughput summaries were committed under `analysis_results/*.csv`. For this report, I regenerated report assets directly from the raw result CSVs so that the tables and figures in the PDF are derived from the committed benchmark data rather than copied manually.

3 Methodology used in this report

3.1 Inputs

The report uses only repository artifacts already committed for Task 1:

- the project description in `README.md`,
- the recorded benchmark summary in `TASK1_SUMMARY.txt`,
- the raw benchmark CSVs in `results/`,
- the benchmark driver logic in `scripts/run_benchmarks.sh`.

3.2 Selection rule

For each dataset and workload:

1. read every result row for an index,
2. compute the arithmetic mean throughput over the three reported repeats,
3. select the row with the highest average throughput for that index,
4. record the chosen row's throughput, standard deviation, index size, and hyperparameter setting.

This matches the rule stated in both the assignment and the committed summary.

3.3 Reported metrics

The report compares:

- **Throughput** in Mops/s for each required workload,
- **Index size** in GiB using the `index_size_bytes` field from the selected best-throughput configuration.

Throughput is the primary Task 1 metric, but index size is included because the assignment context emphasizes that learned indexes can trade space for speed, and the Task 1 request specifically asked for a comparison of throughput *and* index size.

4 Throughput comparison

Table 1 gives the best average throughput for each dataset/workload pair, with the top method in each row shown in bold.

Table 1: Best average throughput (Mops/s) from the committed Task 1 results.

Dataset	Workload	BTree	DynamicPGM	LIPP
Facebook	Lookup-only	1.694	1.797	**358.300**
Facebook	Insert throughput (50/50)	1.059	**6.192**	1.378
Facebook	Lookup after insert (50/50)	1.546	0.471	**349.263**
Facebook	Mixed throughput (10% insert)	1.470	0.849	**13.488**
Facebook	Mixed throughput (90% insert)	1.082	**2.799**	2.158
Books	Lookup-only	1.690	1.947	**361.350**
Books	Insert throughput (50/50)	1.084	**6.124**	2.291
Books	Lookup after insert (50/50)	1.572	0.530	**358.773**
Books	Mixed throughput (10% insert)	1.477	0.935	**17.526**
Books	Mixed throughput (90% insert)	1.058	**2.972**	2.855
OSMC	Lookup-only	2.063	1.804	**307.095**
OSMC	Insert throughput (50/50)	1.032	**5.332**	0.933
OSMC	Lookup after insert (50/50)	1.826	0.455	**253.168**
OSMC	Mixed throughput (10% insert)	1.641	0.948	**10.447**
OSMC	Mixed throughput (90% insert)	1.046	**2.663**	1.621

The most important pattern is immediate:

- **LIPP dominates all read-heavy workloads.** It is the best method for lookup-only, post-insert lookup, and mixed throughput with only 10% inserts on every dataset.
- **DynamicPGM dominates insertion-heavy workloads.** It is the best method for insert throughput in the 50/50 workload and for the mixed workload with 90% inserts on every dataset.
- **BTree is consistent but never best.** It stays in a narrow performance band across workloads, but it does not win any row in Table 1.

The overall averages across the three datasets, shown in Table 2, reinforce the same conclusion.

Table 2: Overall average throughput across the three datasets.

Workload	BTree	DynamicPGM	LIPP
Lookup-only	1.816	1.849	**342.249**
Insert throughput (50/50)	1.058	**5.882**	1.534
Lookup after insert (50/50)	1.648	0.486	**320.401**
Mixed throughput (10% insert)	1.529	0.911	**13.821**
Mixed throughput (90% insert)	1.062	**2.812**	2.211

Several headline comparisons summarize the scale of these differences:

- In the lookup-only workload, LIPP averages 342.249 Mops/s, versus 1.849 Mops/s for DynamicPGM and 1.816 Mops/s for BTree.
- Relative to BTree, LIPP is about **188.5× faster** on average for lookup-only and about **9.0× faster** for the mixed workload with only 10% inserts.
- For 50/50 insert throughput, DynamicPGM averages 5.882 Mops/s, which is about **5.6× faster** than BTree and about **3.8× faster** than LIPP.
- For the 90% insert mixed workload, DynamicPGM still leads at 2.812 Mops/s, ahead of LIPP at 2.211 Mops/s and BTree at 1.062 Mops/s.

Figure 1 visualizes the same throughput results. The log-scale y-axis is necessary because LIPP's lookup throughput is hundreds of Mops/s while the other indexes stay near single-digit Mops/s.

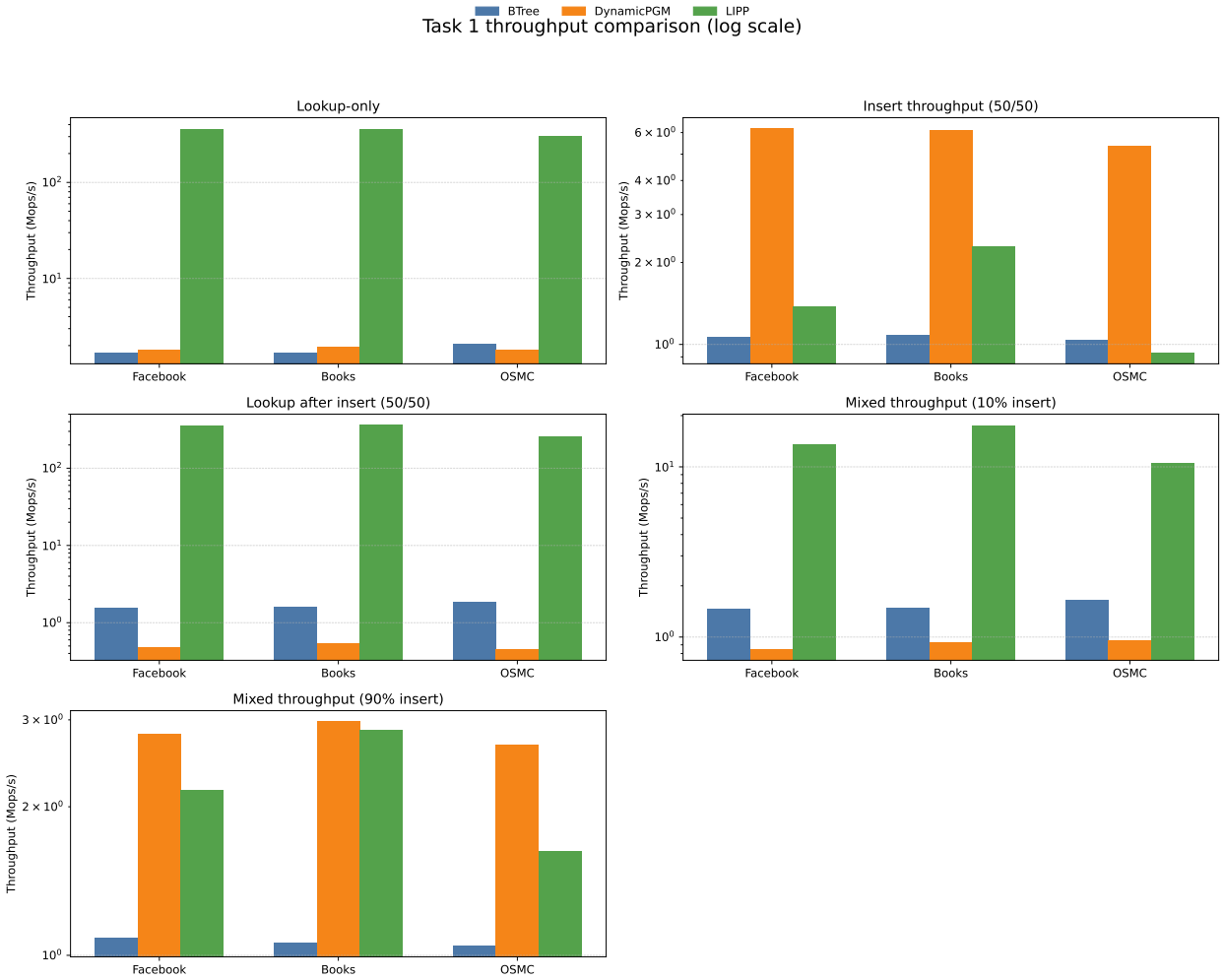

Figure 1: Throughput comparison for all Task 1 workloads and datasets (log scale).

5 Index size comparison

Table 3 compares the size of the selected best-throughput configuration in each dataset/workload pair. Here, the smallest index size in each row is shown in bold.

Table 3: Index size (GiB) of the selected best-throughput configuration.

Dataset	Workload	BTree	DynamicPGM	LIPP
Facebook	Lookup-only	1.73	**1.60**	11.83
Facebook	Insert throughput (50/50)	2.05	**1.58**	11.81
Facebook	Lookup after insert (50/50)	2.05	**1.58**	11.81
Facebook	Mixed throughput (10% insert)	1.82	**1.59**	11.83
Facebook	Mixed throughput (90% insert)	2.34	**1.59**	11.79
Books	Lookup-only	1.73	**1.60**	11.01
Books	Insert throughput (50/50)	2.05	**1.58**	10.98
Books	Lookup after insert (50/50)	2.05	**1.58**	10.98
Books	Mixed throughput (10% insert)	1.82	**1.59**	11.01
Books	Mixed throughput (90% insert)	2.20	**1.59**	10.95
OSMC	Lookup-only	1.73	**1.60**	19.21
OSMC	Insert throughput (50/50)	2.05	**1.58**	19.10
OSMC	Lookup after insert (50/50)	2.05	**1.58**	19.10
OSMC	Mixed throughput (10% insert)	1.82	**1.59**	19.19
OSMC	Mixed throughput (90% insert)	2.20	**1.58**	19.01

The size pattern is much more stable than the throughput pattern:

- **DynamicPGM is always the most compact.** Across all selected configurations it stays near 1.58 GiB to 1.60 GiB.
- **BTree is moderately larger.** It ranges from about 1.73 GiB to 2.34 GiB, averaging 1.98 GiB.
- **LIPP is dramatically larger.** It ranges from about 10.95 GiB to 19.21 GiB, averaging 13.97 GiB.

This means LIPP's speed advantage comes with a substantial memory cost. Averaged across all selected configurations, LIPP is about **8.8× larger** than DynamicPGM and about **7.1× larger** than BTree. The gap is even larger on OSMC: in the lookup-only workload, LIPP is about **12.0× larger** than DynamicPGM.

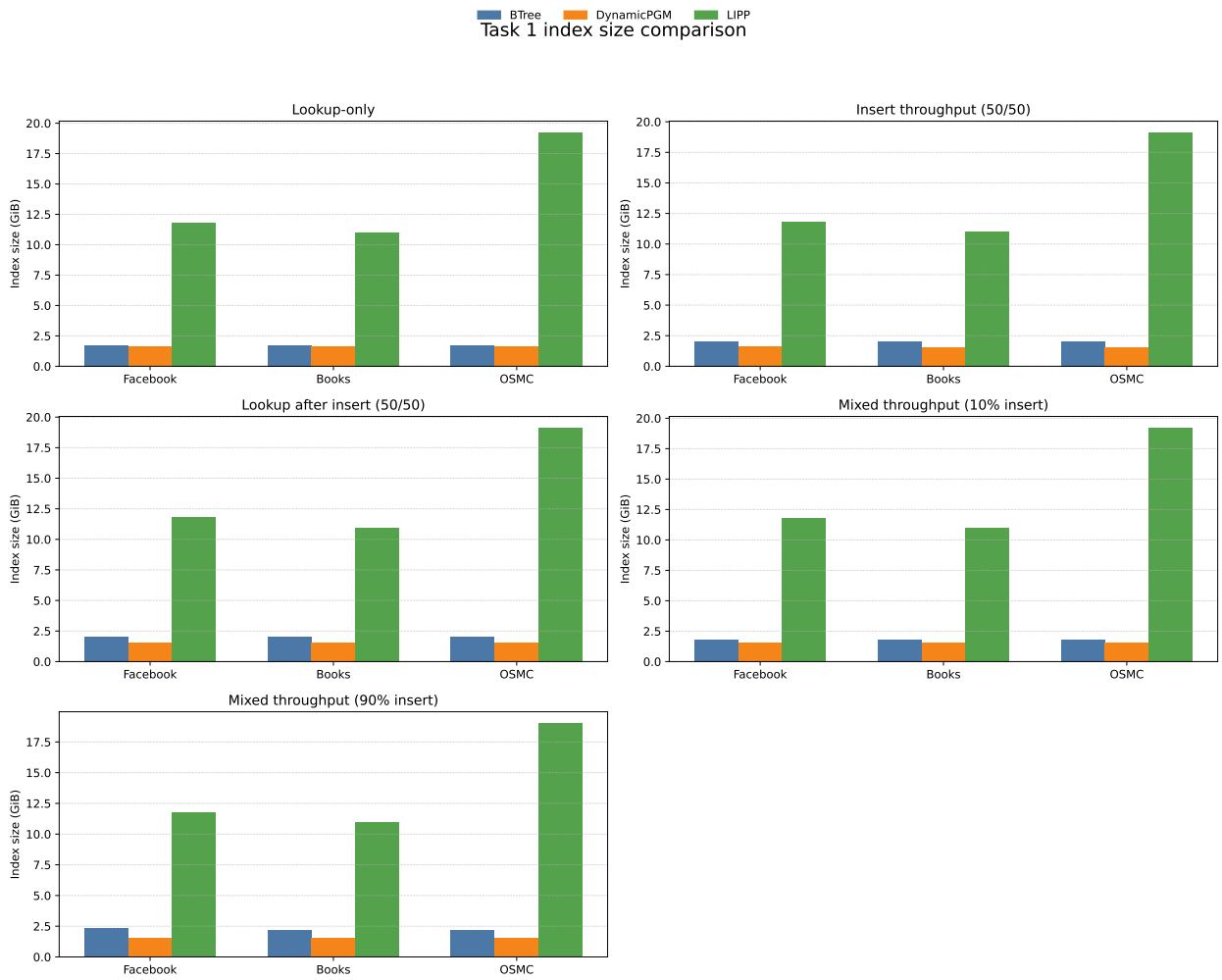

Figure 2: Index size comparison for the selected best-throughput configuration in each workload.

6 Why the performance differs

6.1 Why LIPP wins reads

LIPP is designed to predict precise positions within node arrays. In the repository README, LIPP lookup is described as a model-guided traversal that can return directly from a predicted slot, or recurse only when a collision has already been turned into a child node. That design minimizes the corrective search work at lookup time.

The benchmark results match that expectation strongly:

- lookup-only throughput is above 300 Mops/s on all three datasets,
- post-insert lookup remains above 250 Mops/s on all three datasets,
- even the 10% insert mixed workload stays between 10.447 Mops/s and 17.526 Mops/s, well above the alternatives.

So the empirical story is that when the workload is dominated by lookups, the benefit of direct model-based positioning outweighs the cost of maintaining LIPP's larger structure.

6.2 Why DynamicPGM wins writes

DynamicPGM supports updates using a buffered, amortized strategy rather than forcing each insert to pay the full cost of maintaining exact placement. The README describes this as a logarithmic method with buffers or auxiliary indexes of increasing size. That architecture is exactly the kind of approach that should help insertion-heavy workloads.

Again the results line up with the design:

- DynamicPGM is the winner for insert throughput in the 50/50 workload on every dataset, between 5.332 Mops/s and 6.192 Mops/s.
- DynamicPGM also wins every 90% insert mixed workload, where it reaches 2.663 Mops/s to 2.972 Mops/s.

The downside is visible in the read side: once lookups dominate, DynamicPGM's need for approximate prediction plus corrective searching leaves it far behind LIPP.

6.3 Why BTree sits in the middle

BTree is a classical balanced search tree. It does not have LIPP's extremely fast learned direct-slot lookup, but it also does not depend on DynamicPGM's buffered rebuild-and-merge update strategy. That makes it predictable and relatively stable, but not best-in-class for either extreme.

Across the five workload metrics, BTree remains clustered around:

- roughly 1.7 Mops/s–2.1 Mops/s for lookup-heavy cases,
- roughly 1.0 Mops/s for insert-heavy cases.

That stability may still be attractive as a baseline, but the Task 1 results show that it gives up too much performance relative to the better-specialized learned indexes.

6.4 How dataset properties affect the outcomes

The winner pattern stays consistent across Facebook, Books, and OSMC, which is itself an important finding: the ordering of methods is driven more by workload mix than by the choice of dataset. Still, the dataset does affect the magnitude of the gains:

- LIPP's lookup-only throughput is highest on Books (361.350 Mops/s) and lowest on OSMC (307.095 Mops/s).
- In the post-insert lookup metric, OSMC is also LIPP's weakest dataset (253.168 Mops/s), suggesting that this distribution is harder for LIPP's update-maintained structure than Facebook or Books.
- DynamicPGM's insertion throughput remains strong on all three datasets, but it is slightly lower on OSMC than on Facebook or Books.

The OSMC result also shows the highest run-to-run variability for LIPP after inserts. That does not change the ranking, but it is a useful caution when interpreting averages.

7 Best hyperparameters selected from the recorded runs

Task 1 requires choosing the best hyperparameter setting for BTree and DynamicPGM rather than averaging across all variants. Table 4 records the winning configuration from the committed CSVs for each dataset/workload pair.

Table 4: Best hyperparameter setting selected for each dataset/workload pair.

Dataset / Workload	Best BTree configuration	Best DynamicPGM configuration
Facebook / Lookup-only	LinearSearch / 8	LinearSearch / 16
Facebook / Insert throughput (50/50)	LinearSearch / 8	ExponentialSearch / 256
Facebook / Lookup after insert (50/50)	LinearSearch / 8	ExponentialSearch / 256
Facebook / Mixed throughput (10% insert)	LinearSearch / 8	BinarySearch / 64
Facebook / Mixed throughput (90% insert)	LinearSearch / 6	BinarySearch / 128
Books / Lookup-only	LinearSearch / 8	LinearSearch / 16
Books / Insert throughput (50/50)	LinearSearch / 8	InterpolationSearch / 512
Books / Lookup after insert (50/50)	LinearSearch / 8	InterpolationSearch / 512
Books / Mixed throughput (10% insert)	LinearSearch / 8	LinearSearch / 32
Books / Mixed throughput (90% insert)	LinearSearch / 8	LinearSearch / 32
OSMC / Lookup-only	LinearSearch / 8	LinearSearch / 16
OSMC / Insert throughput (50/50)	LinearSearch / 8	LinearSearch / 1024
OSMC / Lookup after insert (50/50)	LinearSearch / 8	BinarySearch / 1024
OSMC / Mixed throughput (10% insert)	LinearSearch / 8	BinarySearch / 64
OSMC / Mixed throughput (90% insert)	LinearSearch / 8	BinarySearch / 128

Two broad trends emerge:

- BTree almost always prefers `LinearSearch` with value 8, except for the Facebook 90% insert workload where value 6 wins.
- DynamicPGM prefers small values for lookup-only (16 on all datasets), while insertion-heavy workloads prefer larger values such as 32, 64, 128, 256, 512, or 1024 depending on dataset and search method.

This is consistent with the intuition that DynamicPGM can trade lookup precision for better update behavior under insertion-heavy conditions.

8 Detailed Task 1 summary

Task 1 serves as the empirical foundation for the rest of the project. The main point of the milestone is not just to produce bar plots, but to understand the workload-dependent strengths and weaknesses of the baseline indexes before attempting a hybrid design in Task 2.

The detailed summary of what was accomplished is:

1. set up the minimal Task 1 benchmark pipeline from the repository scripts,
2. generated the prescribed workloads for all three datasets,
3. benchmarked BTree, DynamicPGM, and LIPP with three repeats,
4. evaluated multiple BTree and DynamicPGM hyperparameter settings and selected the best-performing one for each case,

5. aggregated results into per-workload summaries,
6. interpreted the measurements in terms of the underlying index designs.

The final Task 1 conclusion is straightforward:

- choose **LIPP** for lookup-dominated workloads when memory overhead is acceptable,
- choose **DynamicPGM** for insertion-dominated workloads,
- use **BTree** as a stable classical baseline rather than the best option for peak throughput.

That conclusion directly motivates the Task 2 project direction: if DynamicPGM is best at absorbing inserts and LIPP is best at serving lookups, then a hybrid design may be able to combine their strengths if the migration or flushing cost can be controlled.

9 Conclusion

Using the committed Task 1 benchmark artifacts in this repository, we compared BTree, DynamicPGM, and LIPP across the Facebook, Books, and OSMC datasets and across the four required workloads. The measured results are highly consistent:

- LIPP is by far the fastest for lookup-heavy workloads,
- DynamicPGM is the fastest for insertion-heavy workloads,
- BTree is consistent but does not win either extreme,
- DynamicPGM is the most space-efficient,
- LIPP achieves its read performance with a much larger index footprint.

These results provide the right baseline for the next stage of the assignment. They show exactly why a hybrid DynamicPGM+LIPP design is attractive: the two learned indexes win different workload regimes, and the remaining challenge is whether their strengths can be combined without paying too much migration overhead.